0286. Walls and Gates

Medium · Graphs

leetcode.com/problems/walls-and-gates

Problem

- Grid: -1 wall, 0 gate, INF (2147483647) empty room. Fill each room with the distance to its nearest gate; unreachable rooms stay INF.

Intuition

- Multi-source BFS from all gates simultaneously: the first time BFS reaches a room is via its nearest gate.

Brute Force

- BFS from each room to find the nearest gate: O((mn)^2).

Optimal Approach

- Enqueue all gates.
- Pop a cell; for each neighbour that is INF, set it to cell + 1 and enqueue.

Worked Example

```
INF  -1  0  INF          3 -1 0 1
INF INF INF  -1    ->    2  2 1 -1
INF  -1 INF  -1          1 -1 2 -1
  0  -1 INF INF          0 -1 3 4
```

Java Solution

```java
public class WallsAndGates {
    static final int INF = Integer.MAX_VALUE;

    public void wallsAndGates(int[][] rooms) {
        int m = rooms.length, n = rooms[0].length;
        Deque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) if (rooms[r][c] == 0) q.add(new int[]{r, c});
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!q.isEmpty()) {
            int[] cur = q.poll();
            for (int[] d : dirs) {
                int r = cur[0] + d[0], c = cur[1] + d[1];
                if (r < 0 || c < 0 || r >= m || c >= n || rooms[r][c] != INF) continue;
                rooms[r][c] = rooms[cur[0]][cur[1]] + 1;
                q.add(new int[]{r, c});
            }
        }
    }
}
```

Complexity

- Time: O(m n). Space: O(m n).

Pitfalls

- Only expand into INF cells; that check doubles as the visited test.